

Slime' s Space Travel

～開発要点書～

2026. 5. 29

日下 拓海

目次

1. 基本情報
2. ゲームの特徴
3. 技術概要
4. システム構成・設計
5. 技術的に工夫した点・問題解決
6. ビルド・リリース
7. まとめ

※本ゲームはデモ版が完成していますが、開発途中のため今後仕様が変更される可能性があります。

1. 基本情報

1-1. 作品情報

ゲームタイトル: Slime' s Space Travel

ジャンル: ステージクリア型 3D アクションゲーム

対応プラットフォーム: Windows / macOS

使用技術: C++、OpenGL（描画）、SDL（音楽、フォント）、Bullet（コリジョン作成）、Assimp（3D モデルロード）など。汎用ゲームエンジンは使用していません。

制作期間: 2026. 3 - 2026. 5（開始から 3 ヶ月経過、開発中）

所要時間数の概数: 300 時間（およその計測値になります。）

制作人数：1人

ゲーム開発における役割：全てのプログラム作成、一部モデルとサウンド作成を含めて全工程を担当しました。

1-2. ゲーム概要

球体地形を探索し、惑星上に散らばっているロケットのパーツを集めたり、敵を倒したりすることで現れるロケットに乗って他の惑星に移動し、願いを叶える星を見つけることを目指すゲームです。





1-3. 開発者情報

開発者名：日下 拓海

所属：弘前大学理工学部電子情報工学科

連絡先：takumi.090414528@gmail.com

1-4. プレイ映像／PV

本作デモ版のプレイ映像です。

<https://youtu.be/5Cc1qw2J5W0>

本作デモ版の PV です。

<https://youtu.be/Nl9kpUEs8NE>

2. ゲームの特徴

2-1. 球体地形を自由に歩き回れる 3D アクション

本作では、一般的な平面地形ではなく、球体状の惑星をステージとして採用しています。プレイヤーは惑星の表面を自由に歩き回り、惑星から突き出した足場や少し離れた足場に飛び乗りながら探索できます。

球体地形ならではの立体的な移動と、惑星を一周するような探索体験が本作の特徴です。

2-2. 楕円地形や緩やかな壁も歩くことができる

球体だけでなく、球を潰したような楕円形の地形や、緩やかな壁に沿った移動にも対応しています。表面だけでなく裏側まで探索できるため、「見えない場所に何かあるかもしれない」という探索の楽しさを生み出しています。

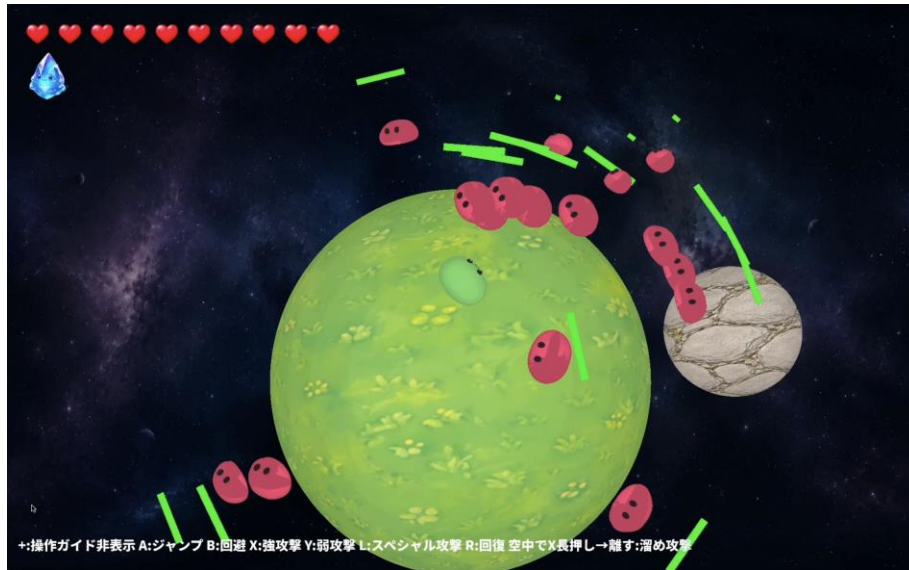


2-3. 弱攻撃と強攻撃でコンボを繋ぎ、敵のガードを破壊する戦闘形式

敵はHPに加えてガードを持っており、弱攻撃や強攻撃を組み合わせたコンボによってガードを破壊できます。

3回目の攻撃を当てることでガードを1つ破壊し、すべてのガードを壊すと敵を空中に打ち上げられます。

打ち上げた敵には空中ため攻撃で大ダメージを与えることができ、球体地形上で敵の位置や攻撃範囲を考えながら戦うアクション性を持たせています。



2-4. ロケットに乗って様々な惑星を旅する

惑星に散らばったロケットパーツを集めたり、敵を倒したりすることでロケットが出現します。ロケットに乗ると次の惑星へ移動でき、ステージ上の背景惑星を眺めながら移動する演出によって、冒険している感覚を高めています。

デモ版では移動ルートは限定的ですが、複数の惑星を旅しているような体験を目指して実装しました。

2-5. 操作方法

ゲームコントローラーの場合（開発者推奨）

Lスティック：移動

Rスティック：視点移動

L：スペシャル攻撃

R：回復

A：ジャンプ

Y：弱攻撃

X：強攻撃

B：回避

キーボードの場合

W/A/S/D: 移動

N: スペシャル攻撃

M: 回復

SPACE: ジャンプ

J: 弱攻撃

K: 強攻撃

U: 回避

3. 技術概要

3-1. 使用技術（言語・ライブラリ）

プログラミング言語: C++/GLSL

グラフィックス API: OpenGL

ウィンドウ管理・入力: GLFW

音楽再生: SDL_mixer

フォント読み込み: SDL_ttf

物理演算・当たり判定: Bullet Physics

3D モデル読み込み: Assimp

YAML ファイル読み込み（データ駆動）: yaml-cpp

算術演算・行列等: GLM

ビルド管理: CMake

※汎用ゲームエンジンは使用せず、OpenGL/GLFW/Bullet Physics などのライブラリを組み合わせ、本作専用の描画・入力・物理・音声システムを実装しました。

3-2. 使用技術選定理由

プログラミング言語: C#でも OpenTK を通して OpenGL を扱うことができますが、本作では C++を選定しました。OpenGL や Bullet Physics などの C/C++系ライブラリを公式 API やサンプルコードに近い形で扱いながら、描画や物理の仕組みを低レイヤーから理解して実装したかったからです。

グラフィックス API: より多くの方にプレイテストをしてもらいたいと考え、Windows/macOS 両方でプレイできる、クロスプラットフォームに対応した OpenGL を選定しました。DirectX も候補にありましたが、macOS 環境でも動作確認しやすい構成にしたかったため、OpenGL を選定しました。

また、OpenGL は学習資料が多く、初学者でも描画の仕組みを理解しながら実装しやすい点も理由です。一方で、現在ではやや古めの技術でもあるため、今後は Vulkan も学習し、より低レイヤーな描画 API での開発にも挑戦したいと考えています。

ウィンドウ管理・入力: SDL で統一することも候補にありましたが、本作は GLFW を選定しました。GLFW は OpenGL の学習用の資料やサンプルで採用されていることが多かったからです。

音楽再生: OpenAL のように 3D 音響に強いライブラリも候補にありましたが、まずは BGM や SE をそのまま再生する機能だけで十分だったため、比較的簡潔なコードで再生、音量調整などができる SDL_mixer を選定しました。今後必要に応じて 3D 音響を取り入れるために OpenAL の導入を検討していきます。

フォント読み込み: FreeType のように細かくフォント制御できるライブラリも候補にありましたが、本作では会話文や UI テキストの表示が主な目的だったため、実装が簡潔で日本語フォントにも対応しやすい SDL_ttf を選定しました。

物理演算・当たり判定: PhysX のような高機能な物理エンジンも候補にありましたが、学習コストや導入コストを考慮し、本作では Bullet Physics を選定しました。

簡易的なコリジョンを自作する選択肢もありましたが、球状地形、突き出した足場、緩やかな壁の上を移動するためには、レイキャストによる地面法線の取得や安定した衝突判定が必要でした。そのため、C++ で扱いやすく、公式サンプルを参考にしながら実装できる Bullet Physics が適していると判断しました。

3D モデル読み込み: インターネット上に公開されている 3D モデルには OBJ や FBX など様々な形式のものがあるため、その複数形式に対応している Assimp が適していると考え選定しました。3D モデル読み込みのライブラリを導入した理由としては、以前の開発でプレイヤーのスライムを正方形にテクスチャを貼っただけの箱として実装していて、スライム感がないことを課題に感じていたことや、球体ステージを作る上で 3D モデルの描画が必須だったことがあります。

YAML ファイル読み込み（データ駆動）：C++でYAMLを扱う際に主流な yaml-cpp を選定しました。以前JSONを用いて開発していた際に、データの一部を一時的に無効化したくてもコメントアウトできず、不便に感じた経験がありました。そのため、コメントアウトや補足コメントを記述でき、手作業で調整しやすいYAMLを採用しました。

TOMLも候補にありましたが、本作ではActor配置やUI情報など階層構造を持つデータを多く扱うため、入れ子構造を表現しやすいYAMLの方が適していると判断しました。

算術演算・行列等：本作ではOpenGLを導入しているため、相性が良いGLMを選定しました。

ビルド管理：Windows / macOS の両方でビルドできる構成にするため、CMake を採用しました。Visual Studio 固有の設定や手書き Makefile に依存せず、外部ライブラリのリンク設定などを CMakeLists.txt にまとめて管理できる点が理由です。

4. システム構成・設計

4-1. 全体構成

本作では、処理の役割ごとにソースコードを分類しています。ゲームオブジェクト、再利用可能な機能、描画、状態管理、システム処理、汎用処理を分けることで、各処理の責務を把握しやすい構成を目指しました。system フォルダは共通システムをまとめていますが、機能追加に応じて責務単位でさらに分割できる構成を意識しています。

src/

└─ actor/	Player、Enemy、Planet などのゲームオブジェクト
└─ component/	Actor に追加する再利用可能な機能
└─ gfx/	3D 描画、UI 描画、Shader、Texture 管理
└─ state/	ゲーム進行状態・UI 表示状態の管理
└─ system/	Audio、Physics、Camera、データ読み込みなどの管理処理
└─ utils/	数学処理などの補助機能

4-2. Component を導入した理由

本作では、Actor に機能を追加する方法として、継承だけでなく Component を導入しました。

例えば「収集できる Actor」を作る場合、Actor を継承して CollectableActor を作る方法があります。しかし、その Actor に「一定時間カメラを向ける機能」も追加したい場合、さらに CollectableFocusableActor のような派生クラスが必要になり、機能の組み合わせが増えるほど継承階層が複雑になります。

一方で Component を用いる場合は、Actor に CollectableComponent や FocusComponent を持たせることで、必要な機能を組み合わせることができます。例えば「収集できる」「フォーカスできる」「破壊できる」といった機能も、それぞれ Component として分離しておけば、Actor 側に必要な Component を持たせるだけで実現できます。

このように、機能を継承で増やすのではなく Component として分離することで、クラス階層を深くしすぎず、再利用しやすい構成を目指しました。

4-3. 継承と Component の使い分け

Component は機能を組み合わせやすい一方で、すべてを Component として表現すると、Component 同士の依存関係が複雑になる場合があります。

例えば Key をすべて Component で表現しようとする、通常の Actor に KeyComponent と CollectableComponent を持たせる形になります。しかし、KeyComponent 内で「取得されたかどうか」を判断したい場合、CollectableComponent の状態を参照する必要があり、Component 同士の依存関係が増えてしまいます。

そのため本作では、Key のようにゲーム内で固有の意味や振る舞いを持つものは Actor を継承した具体的なクラスとして実装し、「取得できる」「一定時間カメラを向ける」といった再利用可能な機能は Component として分離しました。

これにより、Key 固有の処理は Key クラスに、取得可能という共通機能は CollectableComponent に持たせることができ、継承と Component のメリットを両方活かせる構成にしています。

4-4. 設計時に重視したこと

設計時に特に重視したことは、可読性と保守性の向上です。

本作は個人制作ですが、途中から新しい開発者が参加した場合でも、短期間でコードの意図を理解できるようにすることを意識しました。

具体的には、複雑な `if` 文の条件はそのまま書かず、条件の意味を表す変数に置き換えるようにしています。これにより、処理の条件を読んだだけで「何を判定しているのか」が分かるようにしました。

また、関数名を見るだけで内部の処理内容が分かるように命名し、処理の流れを追いやすように関数を分割しています。

例えば、攻撃中の更新処理では、攻撃中の移動、攻撃判定、待機状態への遷移といった処理をそれぞれ関数に分け、`UpdateAttacking` から呼び出す形にしています。これにより、`UpdateAttacking` を読むだけで、攻撃中に行う処理の流れを把握できるようにしました。

さらに、自分でコーディング規約を作成し、規約に沿って実装できているかを定期的に確認しながらリファクタリングを行っています。

規約では、ファイル名・変数名・関数名の命名規則、メンバを変更しない関数の `const` 化、変更されない変数の `const` 化、コンパイル時に決定できる値の `constexpr` 化などを定めています。

これらの取り組みにより、個人制作であっても、後から読み返しやすく、機能追加や修正を行いやすいコードを目指しました。

5. 技術的に工夫した点・問題解決

5-1. 地面法線を用いた球体地形・壁移動

当初は、惑星の中心からプレイヤー位置へのベクトルを計算し、それをプレイヤーの上方向ベクトルとして扱っていました。

この方法であれば、単純な球体地形の表面では自然に立つことができます。しかし、惑星表面から少し浮いた足場や、角度のついた足場へ移動した際に問題が発生しました。上方向ベクトルが常に惑星中心を基準に決まるため、実際に足元にある足場の面に対して垂直に立つことができず、キャラクターが足場に埋まったり、姿勢が不自然になったりしました。

そこで、現在立っている地面の向きを基準にする必要があると考えました。具体

的には、Bullet Physics のレイキャストを足元方向に飛ばし、ヒットした地形の法線ベクトルを取得して、それをプレイヤーの上方向ベクトルとして利用するようにしました。

これにより、球体地形だけでなく、惑星表面から突き出した足場や、緩やかな壁、スロープ状の地形に対しても、その面に沿った自然な姿勢で移動できるようになりました。

5-2. Bullet Physics を用いた接地判定・衝突判定

開発初期の惑星は単純な球体だったため、プレイヤーの位置を惑星半径に合わせて補正することで、擬似的な当たり判定を実装していました。

例えば、プレイヤーが惑星半径より内側に入った場合、位置を惑星表面まで押し戻すような処理です。

しかし、惑星モデルに突き出した足場や複雑な地形を追加すると、この方法では対応できなくなりました。

球体半径だけを基準にしているため、モデル上に存在する足場を当たり判定として扱えず、足場をすり抜けたり、正しく着地できなかったりする問題が発生しました。

そこで、惑星モデルから実際の当たり判定を作成するために Bullet Physics を導入しました。

足元方向へのレイキャストによって接地している地形を判定し、移動時にはスリープ判定を用いることで、壁や足場に対して正しく衝突できるようになりました。

その結果、単純な球体だけでなく、突き出した足場や緩やかな壁を含む複雑な地形上でも、すり抜けを抑えながら安定して移動できるようになりました。

5-3. OpenGL による攻撃範囲表示

戦闘では、プレイヤーが現在どの範囲に攻撃できるのかを直感的に把握できる必要がありました。

特に本作では球体地形や斜面上で戦うため、平面前提の攻撃範囲表示では、地形に沿った範囲として見えにくいという問題がありました。

そこで、攻撃方向・攻撃角度・攻撃距離をもとに、攻撃範囲の頂点を動的に生成して描画するようにしました。

範囲の内側は GL_TRIANGLE_FAN を用いて半透明の扇形として描画し、外周部分は GL_TRIANGLE_STRIP を用いて帯状の太線として描画しています。

また、攻撃範囲の頂点はプレイヤーの正面方向と地面法線を基準に生成しているため、球体地形や斜面上でも現在の地形に沿った形で表示できます。

これにより、プレイヤーは攻撃が届く範囲を視覚的に把握しやすくなり、球体地形上でも敵との距離や攻撃方向を判断しながら戦えるようになりました。

6. ビルド・リリース

本作では、自分の開発環境だけで動作する状態ではなく、他の環境でもプレイテストできる状態を目指しました。

実際に Windows / macOS 向けに、実行ファイル、必要なライブラリ、assets、shaders をまとめた配布フォルダを作成し、GitHub Releases にてゲームデモを公開しています。

実際の Releases ページ

<https://github.com/kusaka0914/SlimesSpaceTravel/releases>

これにより、開発環境以外でも起動確認やテストプレイを行える状態にし、不具合修正や操作性の改善を進めています。

また、プレイテスト後には Google Forms で感想を記入してもらい、プレイヤーの意見を取り入れながら改善を行っています。

7. まとめ

7-1. 制作を通して学んだこと

本作では、C++ / OpenGL / Bullet Physics を用いて、球体地形上を自由に移動できる 3D アクションゲームを開発しました。

本開発を通して、汎用ゲームエンジンを使用せず、描画、物理、入力、音声、データ読み込みなどを自分で組み合わせることで、ゲームが動作する仕組みを低レイヤーから理解することができました。

特に可読性などの設計に力を入れて開発したため、個人開発ではありましたが、チーム開発でも活かせる設計意識を高めることができました。

7-2. 本作での工夫

また、球体地形や壁移動、複雑な接地判定、攻撃範囲表示など、本作特有の課題に対して、地面法線、レイキャスト、スweep判定、OpenGL による動的頂点生成などを用いて解決しました。これにより、特殊な地形上でも自然に移動・戦闘できるゲーム体験を実現しました。

7-3. 今後の改善点

今後は、さらにゲームとしての手触りを高めるため、カメラ挙動、攻撃の演出、敵 AI、ステージ構成を改善していきたいと考えています。

また、安全性や OpenGL の描画最適化、パフォーマンス計測についても理解を深め、より安定したゲーム開発ができるようにしていきたいです。